AHB TO APB BRIDGE

Protocol:

The AHB (Advanced High-performance Bus) to APB (Advanced Peripheral Bus) bridge is a common component in ARM-based systems that connects a high-speed AHB bus to a lower-speed APB bus. This bridge is necessary because AHB and APB have different characteristics and protocols. Here's a high-level overview of how the bridge works:

1. AHB Interface:

- **Master-Slave Relationship:** The AHB interface operates in a master-slave configuration where the AHB master (like a CPU or DMA controller) initiates transactions, and the AHB slave (the AHB to APB bridge) responds.
- **Address Phase:** The master provides the address and control signals during the address phase.
- **Data Phase:** Data is transferred between the master and slave in the data phase. The data phase can be read or write, depending on the type of operation initiated by the master.
- **HREADY Signal:** The bridge monitors the AHB's HREADY signal to determine when to proceed with the transaction.

2. Bridge Operation:

- **Transaction Latching:** The bridge latches the address, control, and write data from the AHB bus when the HREADY signal indicates that the AHB transaction is valid.
- **Protocol conversion:** The bridge translates the AHB signals into the appropriate APB signals. This includes converting burst transactions from AHB into single transactions on the APB, as APB does not support burst operations.
- **Address Mapping:** The bridge maps the AHB address space to the APB address space. It may also adjust the address width if necessary.

3. APB Interface:

- **Setup Phase:** The APB interface uses a setup phase where the address, select, and control signals are driven onto the APB bus.
- **Enable Phase:** In the enable phase, data is transferred either from the APB to the bridge (read) or from the bridge to the APB (write). The PREADY signal is used to indicate when the slave is ready for the transfer.
- **PSEL, PENABLE, PWRITE, and PADDR Signals:** These are the key APB signals. PSEL selects the target peripheral, PENABLE indicates the start of the enable phase, PWRITE determines the direction of the data transfer, and PADDR carries the address.

4. Response Handling:

- **AHB Response:** The bridge generates an appropriate response (OKAY, ERROR, etc.) back to the AHB master based on the outcome of the APB transaction.
- **Data Transfer:** If the AHB master initiates a read, the bridge will place the data from the APB peripheral onto the AHB data bus. For a write, the bridge sends the data from the AHB to the APB peripheral.

5. Timing and Synchronization:

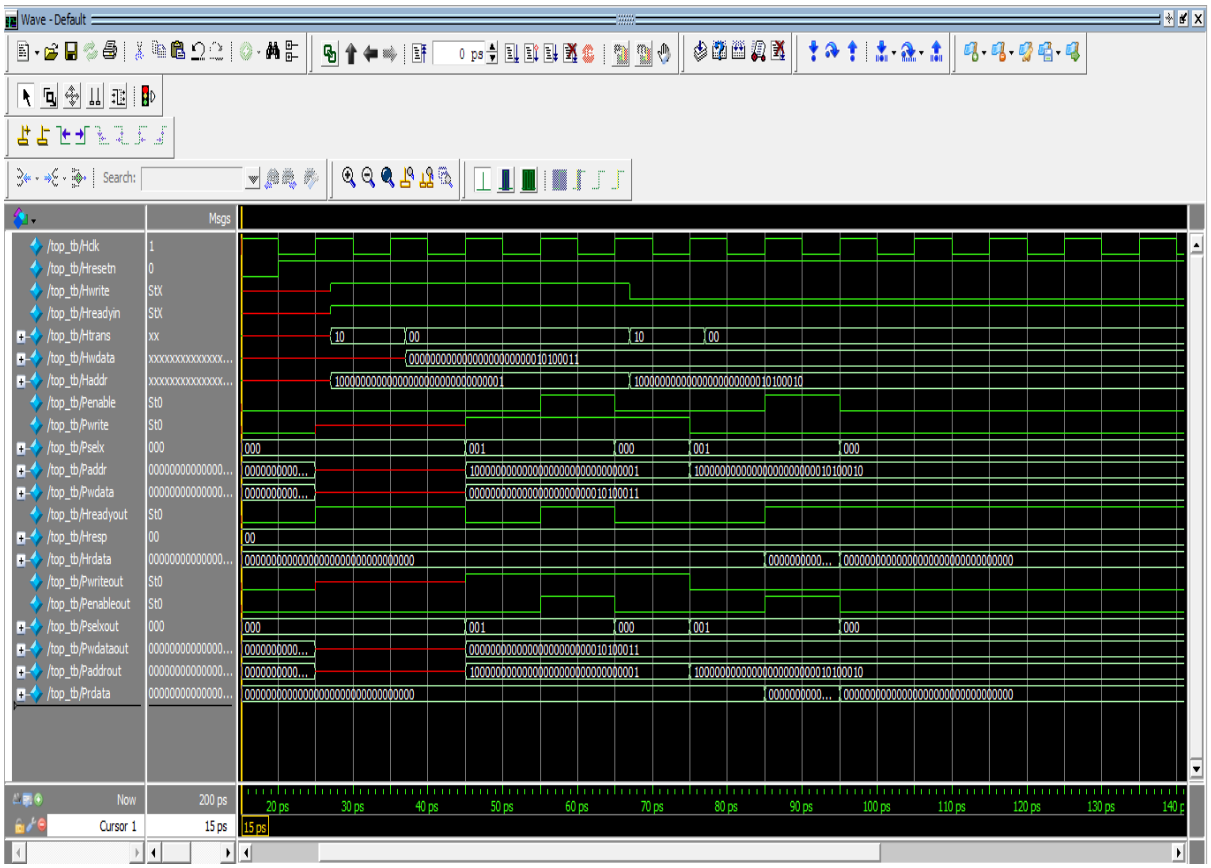
- **Clock Domains:** The AHB and APB may operate in different clock domains, so the bridge might need to handle synchronization between the two buses.
- **Latency:** There is generally a latency of one or more cycles introduced by the bridge as it translates between the AHB and APB protocols.

6. Signals Used:

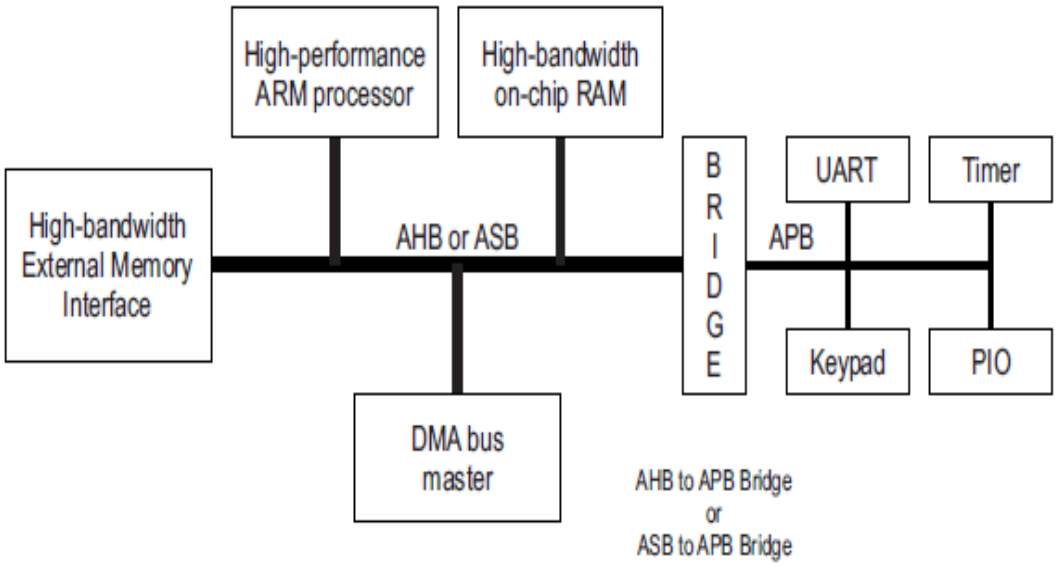
- **AHB Signals:**
  - HADDR (Address)
  - HTRANS (Transaction type)
  - HWRITE (Read/Write control)
  - HSIZE (Transfer size)
  - HBURST (Burst type)
  - HREADY (Ready signal)
  - HWDATA (Write data)
  - HRDATA (Read data)
- **APB Signals:**
  - PADDR (Address)
  - PWRITE (Read/Write control)
  - PSEL (Select signal)
  - PENABLE (Enable signal)
  - PWDATA (Write data)
  - PRDATA (Read data)
  - PREADY (Ready signal)

This AHB to APB bridge protocol ensures smooth communication between a high-performance bus and slower peripheral devices, making it a crucial part of SoC (System on Chip) design.

Simulation waveform:



Block Diagram & Architecture:



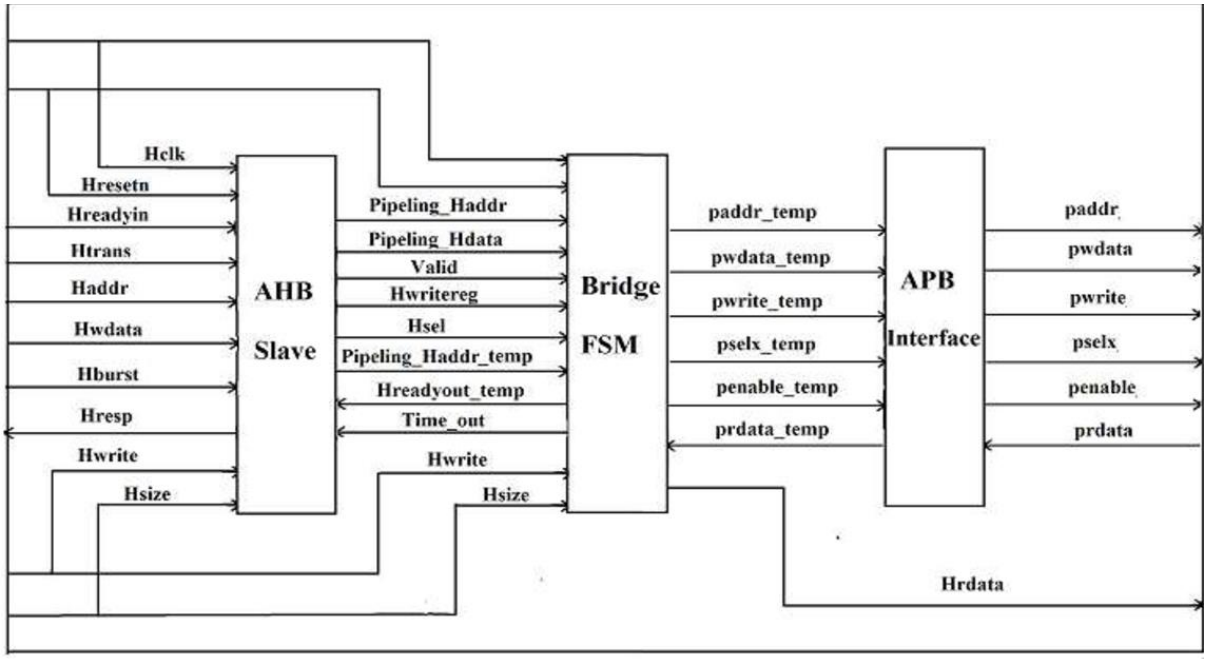
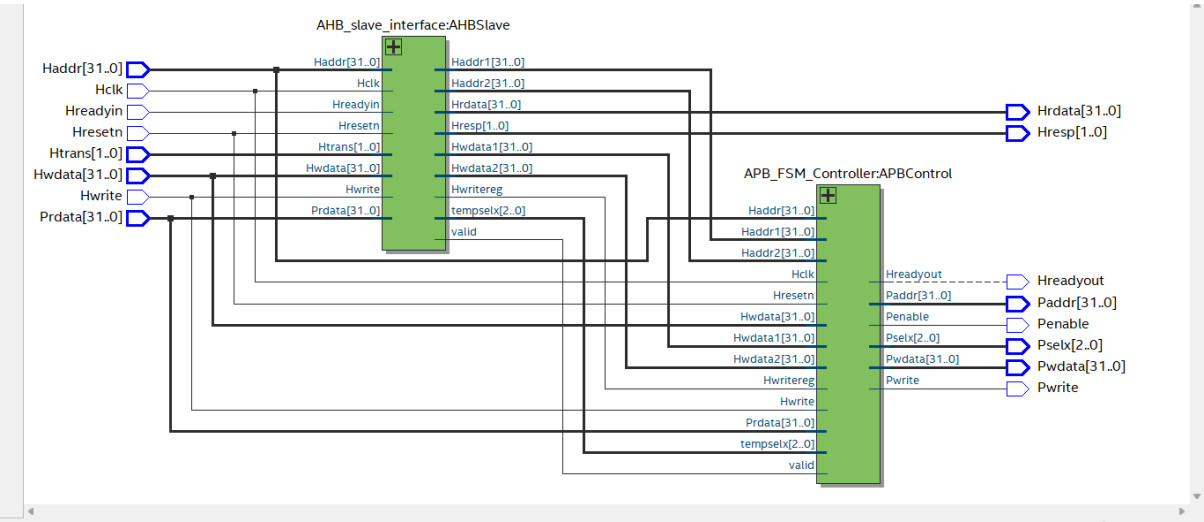


Figure 3.1:. Architecture of AHB to APB Bridge

RTL Synthesis:



Conclusion:

The AHB to APB bridge serves as a critical interface in ARM-based SoCs, enabling efficient communication between high-speed components connected via the AHB and low-power peripherals on the APB. By converting AHB transactions into APB-compatible signals, the bridge ensures seamless data transfer while maintaining the simplicity and low power consumption of the APB. Despite the complexity involved in managing protocol conversion and potential clock domain crossing, the bridge is essential for balancing performance and power efficiency in modern embedded systems. Its design plays a pivotal role in optimizing system operation, particularly in applications requiring both high-speed processing and low-power peripheral management.